

COURSEWORK ASSIGNMENT

Local-Domain Ontology Design, Reasoning and SPARQL

Design and implement an OWL ontology, demonstrate inferred knowledge, and retrieve it through SPARQL.

Course	Semantic Web and Ontologies	Assessment	Individual / Group
Total Marks	100	Bonus	Up to 10 marks
Due Date	31/08/2026	Submission	Online
Lecturer	Prof A TP Silva		

1. Assignment Overview

This assignment provides practical experience in ontology engineering, semantic reasoning and knowledge retrieval. You must select a meaningful local domain, design and implement an ontology representing that domain, and write SPARQL queries that retrieve both explicitly asserted knowledge and knowledge derived through reasoning.

The selected domain should be relevant to a local institution, community, industry, public service or cultural context. It must be sufficiently rich to support meaningful class modelling, property definitions, logical axioms, individuals, inference and query development.

Suggested local domains

- University programmes and course management
- Local healthcare or hospital services
- Sri Lankan tourism and cultural heritage
- Public transport and route information
- Agriculture and crop management
- Disaster management services
- Local-government services
- Traditional food and cuisine
- Wildlife and environmental conservation
- Small and medium-sized enterprises

2. Learning Outcomes

1. Analyse a real-world domain and identify its important concepts, relationships and constraints.
2. Apply ontology-engineering principles to create a structured and reusable knowledge model.
3. Represent domain knowledge using RDF, RDFS and OWL constructs.
4. Define classes, subclasses, object properties, data properties, individuals and restrictions.
5. Use a reasoner to derive new knowledge from ontology axioms.
6. Develop SPARQL queries that retrieve asserted and inferred knowledge.
7. Optionally integrate an ontology with a simple software application.

3. Assignment Tasks

Task 1 — Domain Selection and Requirements Analysis (10 marks)

Describe the selected domain and establish the requirements that will guide the ontology design. Include:

- A clear description of the selected domain
- The purpose and expected users of the ontology
- The scope and boundaries of the ontology
- At least five competency questions that the ontology should answer

Example competency questions include: Which lecturers teach postgraduate courses? Which tourist attractions are located in the Central Province? Which crops are suitable for regions with high rainfall? Which bus routes connect two selected cities?

Task 2 — Ontology Design (20 marks)

Prepare a conceptual model for the domain. Your design must show:

- Main classes and subclasses
- Object properties and data properties
- Property domains and ranges
- Representative individuals
- The class hierarchy
- Important restrictions and logical relationships

Include a diagram showing the principal concepts and relationships. Protégé OntoGraf, WebVOWL or another appropriate visualisation tool may be used.

Task 3 — Ontology Development (30 marks)

Implement the ontology using Protégé or another suitable environment. The ontology must also include an appropriate subclass hierarchy, clearly specified domains and ranges, and labels and descriptions for major ontology elements.

Use only OWL features that are logically appropriate for the domain. Suitable features may include equivalent or disjoint classes, inverse properties, transitive or symmetric properties,

functional properties, property chains, cardinality restrictions, and existential or universal restrictions.

Task 4 — Reasoning and Inference (15 marks)

Use a suitable reasoner, such as HermiT, Pellet, FaCT++ or the reasoning facilities of the selected framework. You must:

1. Check whether the ontology is logically consistent.
2. Identify at least five facts inferred from the ontology.
3. Explain which axioms or property definitions produced each inference.
4. Provide evidence of the inferred results using screenshots or query outputs.

Important: An inferred fact must not be directly asserted in the ontology. For example, if AcademicStaff is a subclass of Employee and DrPerera is an AcademicStaff member, a reasoner can infer that DrPerera is also an Employee.

Task 5 — SPARQL Query Development (15 marks)

Develop at least eight SPARQL queries. The query set must include:

- Two basic SELECT queries
- One query using FILTER
- One query using OPTIONAL
- One query using aggregation such as COUNT, AVG, MIN or MAX
- One query using grouping or ordering
- Two queries that retrieve knowledge produced through reasoning

For every query, provide the competency question addressed, the SPARQL code, the result, a short interpretation, and a statement indicating whether the result uses asserted or inferred knowledge.

Technical note: SPARQL does not automatically perform OWL reasoning in every system. Inferred knowledge must therefore be materialised by a reasoner or queried through a repository that supports an appropriate entailment regime.

Task 6 — Evaluation and Documentation (10 marks)

Evaluate the completed ontology with respect to:

- Logical consistency
- Correctness of the class hierarchy
- Coverage of the competency questions
- Quality of naming and documentation
- Reusability and extensibility
- Current limitations and possible future improvements

4. Bonus Task — Application Integration (Up to 10 Marks)

For bonus marks, develop a simple front-end application that interacts with the ontology. Apache Jena, Owlready2, RDFLib, SPARQLWrapper, a GraphDB API or another suitable Semantic Web framework may be used.

Minimum application capability

The application should allow the user to perform at least two of the following:

- Search for individuals in the ontology
- Run predefined SPARQL queries
- Enter criteria and retrieve matching knowledge
- Display inferred classifications or relationships
- Add a new individual with suitable validation
- View selected classes, properties and relationships

Bonus evidence

- Complete source code
- Setup and execution instructions
- Screenshots of the working application
- A short explanation of the architecture
- Evidence that the application communicates with the ontology
- Evidence that at least one displayed result depends on reasoning or inferred knowledge

Bonus marks will be awarded for functionality, usability, semantic correctness, code quality and the quality of integration with the ontology.

5. Required Deliverables

1. Ontology file in .owl, .rdf or .ttl format
 2. Reasoning evidence, including screenshots or exported results
 3. Ontology diagram included in the report
 4. Bonus application source code and setup instructions, if attempted
 5. Short presentation or live demonstration, if required by the examiner
- Limitations and future improvements
 - Conclusion
 - References

6. Marking Scheme

Assessment component	Marks
Domain selection and requirements analysis	10

Ontology design	20
Ontology implementation	30
Reasoning and inference	15
SPARQL queries and results	15
Documentation /presentation	10
TOTAL	100
Application integration bonus	Up to 10

7. General Instructions

- Use meaningful and consistent names for ontology elements.
- Apply a consistent namespace and URI-naming convention.
- Avoid unnecessary duplication of classes, properties and individuals.
- Use labels and comments to improve readability and maintainability.
- Do not add OWL features merely to satisfy the required count; every modelling decision must be justifiable.
- Test every SPARQL query before submission.
- Clearly distinguish asserted knowledge from inferred knowledge.
- Cite all external datasets, vocabularies, ontologies and information sources.
- Clearly identify and explain any elements reused or adapted from an existing ontology.
- Submitting an existing ontology without substantial original development is not acceptable.
- Every student or group member must be able to explain and demonstrate the submitted work.

8. Academic Integrity

The submitted ontology, queries, report and application must represent the student's or group's own work. External resources, code, datasets, ontologies and AI-assisted content must be acknowledged in accordance with institutional guidelines.